

Algorithmique et Programmation orientée-objet

Ch.5 – Interfaces, Classes abstraites et internes

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

Table des Matières

- **Interfaces**

- Déclaration, Implémentation
- Polymorphisme, héritage
- Comparaison d'objets
- Interfaces fonctionnelles, lambdas
- Gestion d'événements
- Itérateur

- **Classes abstraites**

- Application : timer

- **Classes internes**

- Classes internes et imbriquées
- Classes locales et anonymes

Interfaces

- **Introduction**

- Liste de méthodes **abstraites**
 - juste les signatures, pas d'implémentation
- Définit un contrat de service

```
public interface Attackable {  
    void takeDamage(int damage);  
}
```

```
public interface AttackCapability {  
    void attack(Attackable target);  
}
```

Interfaces

- Exemple

```
public class Farmer
    extends GameCharacter
    implements Attackable {

    public Farmer(String name)
        super(name);
    }

    @Override
    public void takeDamage(int damage) {
        health -= damage;
        say("aïeuh");
    }
}
```



```
Farmer farmer = new Farmer("Maggot");
Warrior warrior1 =
    new Warrior("Gothmog");
Warrior warrior2 =
    new Warrior("Aragorn");

warrior1.attack(farmer);
warrior2.attack(warrior1);
```

```
public class Warrior
    extends GameCharacter
    implements Attackable, AttackCapability {

    public Warrior(String name) {
        super(name);
    }

    @Override
    public void takeDamage(int damage)
        health -= damage / 3;
        say("même pas mal");
    }

    @Override
    public void attack(Attackable target) {
        target.takeDamage(10);
    }
}
```



Interfaces

- **Utilité**

Abstraire un
contrat de
service

Définir un
nouveau **type**

*polymorphisme
d'inclusion, sans
héritage*

Manipuler des
fonctions

*approcher le
style de la
programmation
fonctionnelle*

Remplacer
l'**héritage**
multiple

Mieux
organiser son
code

Interfaces

- **Déclaration**

- Mot-clé **interface**

- méthodes abstraites publiques
 - attributs de classe (**static public final**)
 - méthodes par défaut (**default**)
 - méthodes de classe (**static public**)

≥ Java 8

méthodes privées

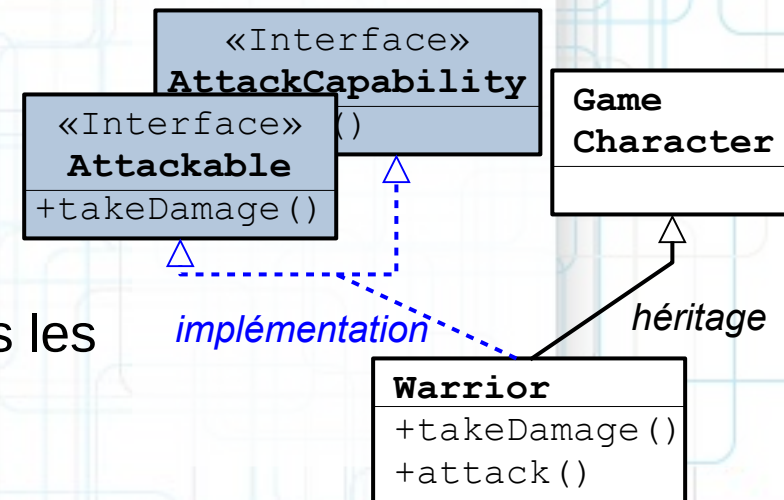
≥ Java 9

- **Implémentation**

- Mot-clé **implements**

- une ou plusieurs interfaces

- Classe doit fournir implémentation de toutes les méthodes définies par les interfaces



Interfaces

- **Convention de nommage**

- Un nom d'interface est un identifiant. Il suit la **notation chameau**, en commençant par une majuscule.
- Les noms d'interfaces sont généralement
 - des **adjectifs exprimant une "capacité de"** (suffixe "able").
Par exemple : Cloneable, Serializable, Comparable, Resizable, Readable
 - des **noms de types**. Par exemple : List, Collection, Map

- **Annotation @Override**

- Informe le compilateur de la volonté de définir une méthode
- Permet de détecter des erreurs de nom / signature

Table des Matières

- **Interfaces**

- Déclaration, Implémentation
- Polymorphisme, héritage
- Comparaison d'objets
- Interfaces fonctionnelles, lambdas
- Gestion d'événements
- Itérateur

- **Classes abstraites**

- Application : timer

- **Classes internes**

- Classes internes et imbriquées
- Classes locales et anonymes

Interfaces

- **Polymorphisme**

- Le type d'une référence polymorphique (type déclaré) peut être une interface. Une telle référence peut désigner tout objet issu d'une classe implémentant cette interface.
- Si la classe **C** implémente l'interface **I**, alors on peut aussi noter **C ▷ I** car **C** est considéré comme un sous-type de **I**.

- **Opérateur instanceof**

- `ref instanceof I` $\Leftrightarrow$ type réel `▷ I`

```
Object ref = new Farmer();  
System.out.println(ref instanceof Attackable);  
System.out.println(ref instanceof AttackCapacity);
```

```
true  
false
```

- **Transtypage**

- Transtypage possible de/vers interface. (/!\ règles Ch. 4)
- Opérateur de transtypage peut également être utilisé pour convertir explicitement vers une interface.

Interfaces

- **Héritage entre interfaces**

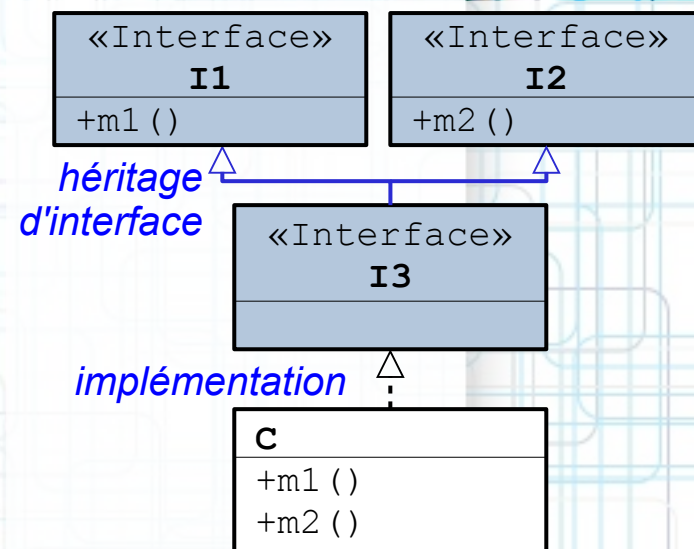
- Il est possible de définir une nouvelle interface par **héritage** d'une ou plusieurs interfaces existantes (mot-clé **extends**).
- L'interface résultante est l'union des définitions des interfaces parent.
- On peut noter **I3 ▷ I1** et **I3 ▷ I2**

```
public interface I1 {
    void m1 ();
}

public interface I2 {
    void m2 ();
}

public interface I3 extends I1, I2 { }

public class C implements I3 {
    public void m1 () { ... }
    public void m2 () { ... }
}
```



Interfaces

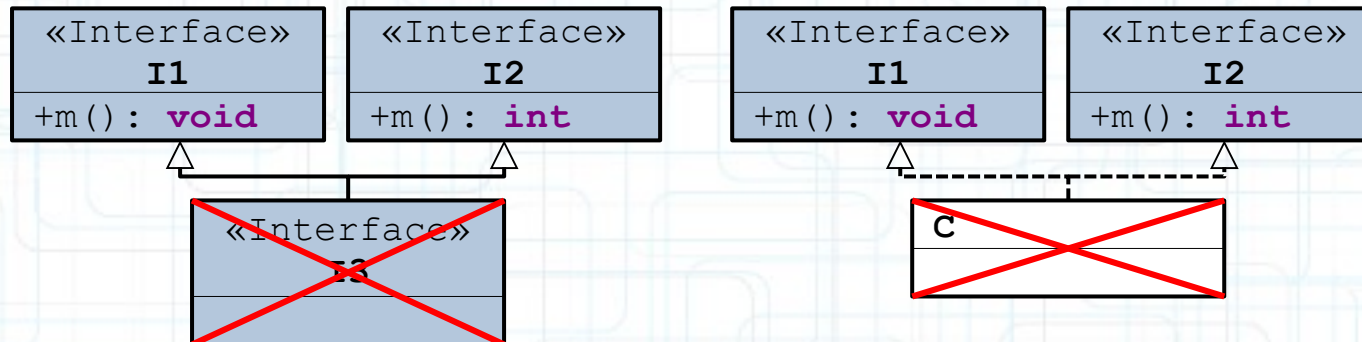
- **Héritage multiple**

- L'héritage multiple peut être approché en Java

- classe implémentant plusieurs interfaces
- interface héritant de plusieurs interfaces

- **MAIS**

- si deux interfaces définissent une **méthode de même nom et signature mais de types retour différents**, il est impossible de les implémenter / hériter simultanément !



≥ Java 8

- si deux interfaces définissent la **même méthode par défaut**, il n'est possible de les implémenter simultanément que si la méthode est re-définie.

Table des Matières

- **Interfaces**

- Déclaration, Implémentation
- Polymorphisme, héritage
- Comparaison d'objets
- Interfaces fonctionnelles, lambdas
- Gestion d'événements
- Itérateur

- **Classes abstraites**

- Application : timer

- **Classes internes**

- Classes internes et imbriquées
- Classes locales anonymes

Comparaison d'objets

- **Comparaison d'objets**

- Java ne supporte pas la surcharge des opérateurs : $<$, $<=$, $>$ et $>=$ ne s'appliquent que sur les types primitifs numériques !
- Une autre approche est utilisée pour permettre la comparaison de n'importe quel type d'objet : l'implémentation de l'**interface Comparable**.

```
public interface Comparable<T> {  
    public int compareTo(T other);  
}
```

- La méthode `compareTo()` doit respecter la convention suivante : `x.compareTo(y)` retourne
 - valeur < 0 si $x < y$
 - valeur 0 si $x = y$
 - valeur > 0 si $x > y$

Comparaison d'objets

- **Relation d'ordre total**

- L'interface `Comparable` définit une **relation d'ordre total** sur les objets d'une classe (et de ses sous-classes).
- Pour rappel, une relation binaire $R \subseteq E \times E$ est une relation d'ordre total si elle satisfait les propriétés suivantes
 - Anti-symétrie $\forall x, y \in E, (x R y) \text{ et } (y R x) \Leftrightarrow x = y$
 - Transitivité $\forall x, y, z \in E, (x R y) \text{ et } (y R z) \Rightarrow x R z$
 - Réflexivité $\forall x \in E, x R x$
 - Totalité $\forall x, y \in E, (x R y) \text{ ou } (y R x)$
- Elle devrait idéalement être cohérente avec `equals()` autrement dit $x.\text{equals}(y) \Leftrightarrow (x.\text{compareTo}(y) = 0)$

Comparaison d'objets

- **Exemple**

- Comparaison d'instances de la classe Point

```
public class Point implements Comparable<Point> {  
    public final double x, y;  
    public Point(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
    public int compareTo(Point other) {  
        int cmpX = Double.compare(x, other.x);  
        if (cmpX != 0)  
            return cmpX;  
        return Double.compare(y, other.y);  
    }  
    public boolean equals(Object other) {  
        if (other == null || !(other instanceof Point))  
            return false;  
        return (x == this.x) && (y == this.y);  
    }  
}
```

Comparaison d'objets

- **Tri et recherche**

- La bibliothèque Java fournit dans la classe `Arrays` (`java.util`) des méthodes de classe permettant le tri ou la recherche dichotomique. Les objets doivent supporter l'interface `Comparable`.
- La méthode `sort()` permet de trier *en place* le contenu d'un tableau.

```
public static void sort(Object[] a);
```

```
String[] prenom = { "Nestor", "Archibald", "Hugolin",  
                   "Ernest" };
```

```
Arrays.sort(prenom);
```

- La méthode `binarySearch()` permet d'effectuer une recherche dans un tableau trié.

```
public static int binarySearch(Object[] a, Object key);
```

```
Arrays.binarySearch(prenom, "Hugolin");
```

```
Arrays.binarySearch(prenom, "Hector");
```

2

-1

Comparaison d'objets

- **Ordre non-naturel**

- L'ordre défini avec Comparable est l'**ordre naturel** de la classe.
- Il est possible de définir d'autres relations d'ordre sur des objets à l'aide de l'interface fonctionnelle Comparator. Cela peut être nécessaire si l'on veut par exemple ordonner des objets selon différents critères.

```
public interface Comparator<T> {  
    int compare(T e1, T e2);  
}
```

- Des variantes des méthodes sort() et binarySearch() peuvent exploiter cette interface.

```
static <T> void sort(T[] a, Comparator<? super T> c);  
static <T> int binarySearch(T[] a, T key,  
                           Comparator<? super T> c);
```

```
Integer[] t = { -2, -1, 0, 1, 4, 5, 17 };  
Arrays.sort(t, (a,b) -> b.compareTo(a));
```

Table des Matières

- **Interfaces**
 - Déclaration, Implémentation
 - Polymorphisme, héritage
 - Comparaison d'objets
 - Interfaces fonctionnelles, lambdas
 - Gestion d'événements
 - Itérateur
- **Classes abstraites**
 - Application : timer
- **Classes internes**
 - Classes internes et imbriquées
 - Classes locales et anonymes

Interfaces fonctionnelles

- **Méthode passée en argument**
 - Il n'existe **pas de type méthode ou type fonction** en Java. Il n'est donc pas possible de manipuler des méthodes et de les donner en argument d'autres méthodes.
 - Les **interfaces fonctionnelles** permettent d'émuler ce comportement. Une interface fonctionnelle est une interface qui ne définit qu'une seule méthode.
 - Exemple

```
public interface MathFunction {  
    double eval(double x);  
}
```

Note : les méthodes et fonctions ne sont pas des *éléments de première classe* du langage Java.

Interfaces fonctionnelles

- Interface fonctionnelle

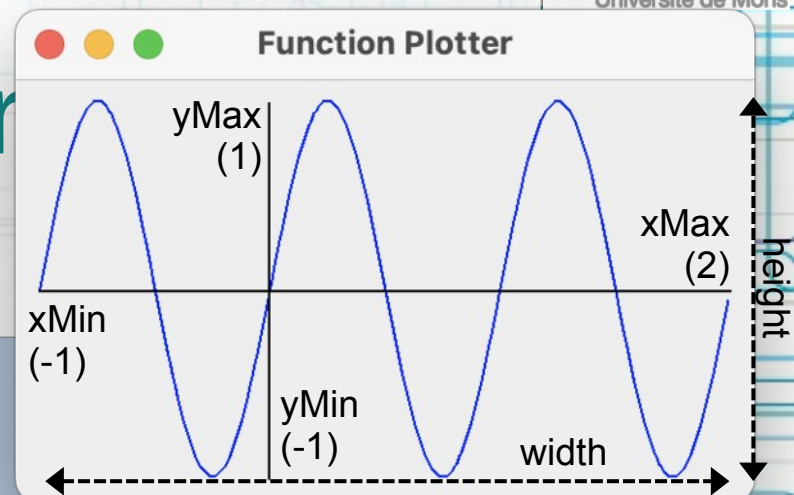
```
public interface MathFunction {
    double eval(double x);
}
```

```
public class Sinus implements MathFunction {
    public double eval(double x) {
        return Math.sin(2 * Math.PI * x);
    }
}
```

```
public class Sqr implements MathFunction {
    public double eval(double x) {
        return x * x;
    }
}
```

```
public static void plot(MathFunction f) {
    for (int i = 0; i < width; i++) {
        double x = xMin + (i * (xMax - xMin) / width);
        double y = f.eval(x) * height / (yMax - yMin) + height / 2;
        graphics.dessinerPoint(i, (int) Math.round(y));
    }
}
```

```
plot(new Sinus());
```



Interfaces fonctionnelles

- Interfaces fonctionnelles

- La bibliothèque Java fournit des définitions d'interfaces fonctionnelles (package `java.util.function`), par exemple pour modéliser des fonctions à 1 ou 2 variables ou des prédicats.

```
@FunctionalInterface  
public interface Function<T,R> {  
    R apply(T t);  
}
```

```
@FunctionalInterface  
public interface BiFunction<T,U,R> {  
    R eval(T t, U u);  
}
```

```
@FunctionalInterface  
public interface Predicate<T> {  
    boolean test(T t);  
}
```

Paramètres de type
(polymorphisme paramétrique)

Annotation `@FunctionalInterface`

communique au compilateur intention
de définir interface fonctionnelle
⇒ vérifie une seule méthode définie.

Interfaces fonctionnelles

- **Expressions lambda**

- Java supporte un mécanisme appelé « **expression lambda** » permettant d'exprimer facilement des fonctions anonymes.

```
Function<Double, Double> f = x -> x * x;  
BiFunction<Double, Double, Double> g = (x, y) -> x * y;
```

- Les lambdas sont en fait des « *sucres syntaxiques* » pour implémenter des interfaces.
- La forme générale des expressions lambda est

$(T_1 \text{ arg}_1, T_2 \text{ arg}_2, \dots, T_N \text{ arg}_N) \rightarrow \text{expression} \mid \text{bloc}$

- types $T_1, \dots, T_N$
 - généralement omis car peuvent souvent être inférés
 - dans ce cas, les parenthèses peuvent être omises si $N=1$.
- corps : une simple expression ou un bloc composé d'instructions (y compris **return**).

Interfaces fonctionnelles

- **Références à des méthodes**

- Lorsqu'il s'agit d'utiliser une **référence vers une méthode existante**, il est possible de donner directement son nom plutôt que de passer par la définition d'une expression lambda.
- Afin de distinguer une référence vers une méthode de l'appel de cette méthode, la syntaxe du langage utilise « :: » comme séparateur plutôt que le point.

```
double integrate (Function<Double, Double> f,  
                 double a, double b) {  
    /* ... */  
}  
double result = integrate(Math::sin, 0, Math.PI);
```

→ méthode de classe

```
Consumer<String> c = System.out::println;  
c.accept("Hello, function");
```

→ méthode d'instance

Interfaces fonctionnelles

- **Style de programmation fonctionnel**
 - Manipuler des séquences de données comme des « *flux* » permet de donner un style fonctionnel à la programmation en Java.
 - L'interface `Stream`, au coeur de cette approche, permet d'appliquer une séquence d'opérations à un flux.

```
int[] t = {1, 4, -1, 0, 17, -2, 5};  
int sum = Arrays.stream(t).sum();
```

```
Arrays  
    .stream(t)  
    .filter(x -> x > 0)  
    .map(x -> 2 * x)  
    .sorted()  
    .forEach(System.out::println);
```

Ne laisse passer que les éléments qui satisfont le prédicat

Remplace chaque élément par son image par la fonction

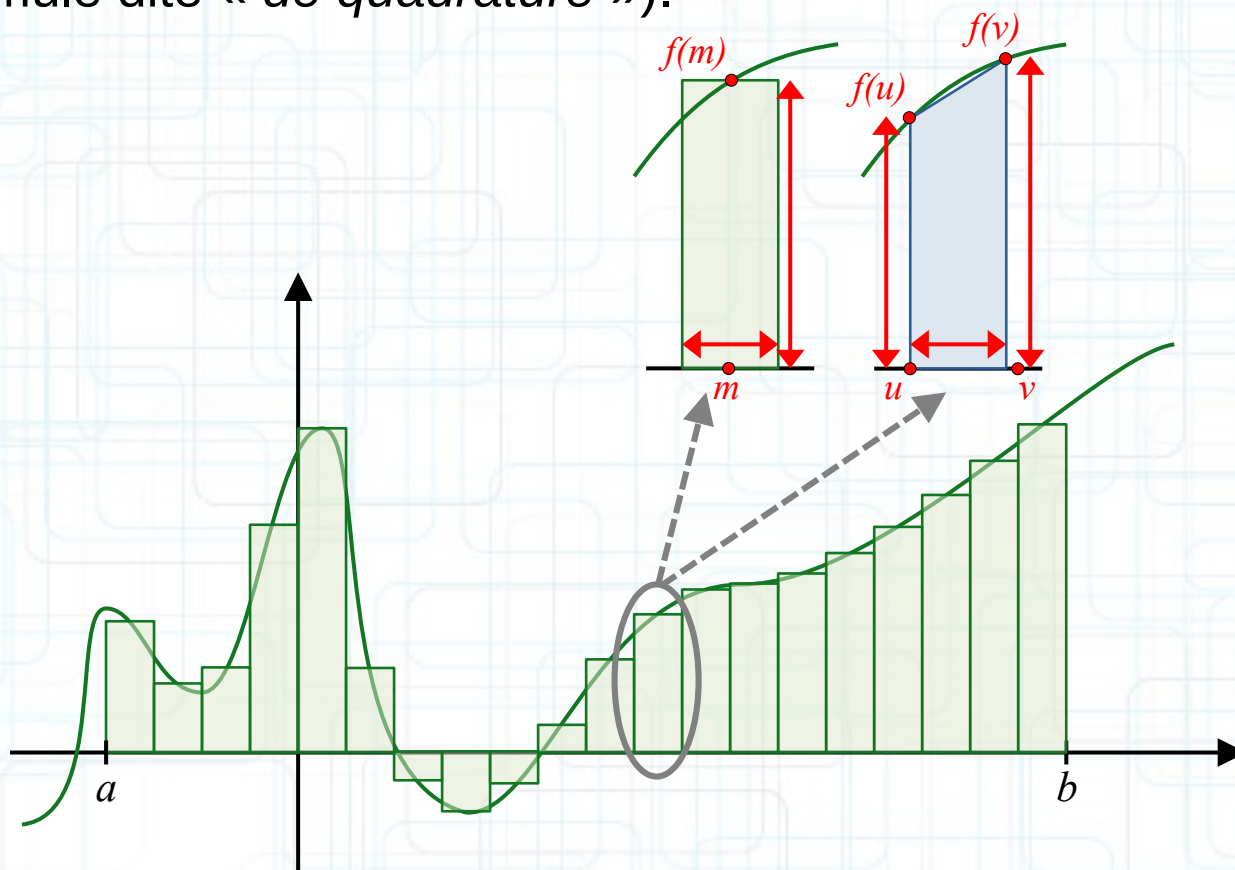
Trie les éléments selon leur ordre naturel

Appelle une méthode sur chaque élément

Interfaces fonctionnelles

- **Exercice**

- En utilisant les interfaces, implémentez un **algorithme d'intégration numérique** qui approxime l'intégrale d'une fonction $f(x)$ sur un intervalle $[a, b]$ par une somme de surfaces (formule dite « *de quadrature* »).



Interfaces fonctionnelles

- **Exercice**
 - **Contraintes**
 - La fonction et les bornes sont des paramètres
 - Différentes méthodes d'approximations peuvent être utilisées (rectangles, trapèzes, Simpson). On doit pouvoir facilement en ajouter une nouvelle sans changer l'implémentation.
 - **Test**
 - Tester sur des fonctions dont la primitive est connue. Par exemple : x , $\sin(x)$, x^2 , ...
 - Comparer les résultats obtenus avec les différentes méthodes (rectangle, trapèze et Simpson). Faire varier le nombre d'intervalles.

Table des Matières

- **Interfaces**

- Déclaration, Implémentation
- Polymorphisme, héritage
- Comparaison d'objets
- Interfaces fonctionnelles, lambdas
- Gestion d'événements
- Itérateur

- **Classes abstraites**

- Application : timer

- **Classes internes**

- Classes internes et imbriquées
- Classes locales et anonymes

Gestion d'événements

- **Gestion d'événements**
 - Une autre application des interfaces est dans la gestion d'événements, au travers de **fonctions « callback »**.
 - Une *callback* est une fonction transmise à un processus et qui est appelée lors de la survenance d'un événement.
 - Exemples d'événements
 - pression d'un bouton ou clic de souris dans une GUI
 - réception d'un message
 - notification de fin d'un traitement
 - expiration d'un timer
 - ...

Gestion d'événements

- Gestion d'événements

```
public interface MouseListener {
    void mouseClicked(MouseEvent e);
    void mouseEntered(MouseEvent e);
    void mouseExited(MouseEvent e);
    void mousePressed(MouseEvent e);
    void mouseReleased(MouseEvent e);
}
```

```
public class MyMouseListener
implements MouseListener
{
    public void mouseClicked(MouseEvent e) {
        System.out.println("Bouton pressé");
    }
    public void mouseEntered(MouseEvent e) { /* ... */ }
    public void mouseExited(MouseEvent e) { /* ... */ }
    public void mousePressed(MouseEvent e) { /* ... */ }
    public void mouseReleased(MouseEvent e) { /* ... */ }
}
```

```
/* ... */
panel.addMouseListener(new MyMouseListener());
/* ... */
```

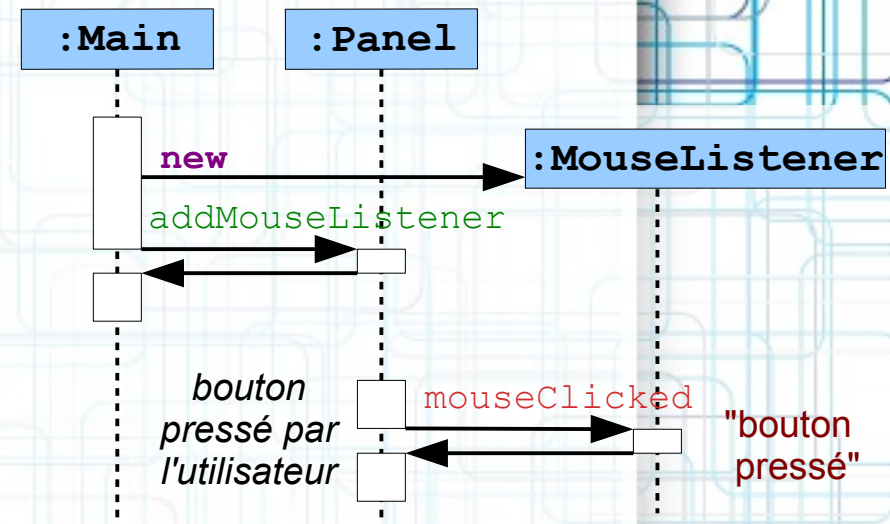


Table des Matières

- **Interfaces**

- Déclaration, Implémentation
- Polymorphisme, héritage
- Comparaison d'objets
- Interfaces fonctionnelles, lambdas
- Gestion d'événements
- Itérateur

- **Classes abstraites**

- Application : timer

- **Classes internes**

- Classes internes et imbriquées
- Classes locales et anonymes

Itérateur

- **Contrat d'itérateur**

- Un itérateur est un objet qui permet le **parcours d'une collection d'éléments** sans révéler la structure interne de leur stockage.
- L'interface `Iterator` définit les méthodes d'un contrat d'itérateur.

```
public interface Iterator<T> {  
    boolean hasNext();  
    T next();  
}
```

Retourne **true** s'il y a encore au moins un élément

Retourne l'élément actuel et fait avancer l'itérateur

- Exemple d'une interaction typique avec un itérateur.

```
Iterator<String> iter = /* ... */  
while (iter.hasNext())  
    System.out.println(iter.next());
```

Itérateur

- Itérateur pour tableau

```
public class ArrayIterator<T> implements Iterator<T> {  
  
    private final T[] tab;  
    private int index;  
  
    public ArrayIterator(T[] tab) {  
        this.tab = tab;  
        index = 0;  
    }  
  
    public boolean hasNext() {  
        return index < tab.length;  
    }  
  
    public T next() {  
        if (index ≥ tab.length)  
            throw new NoSuchElementException();  
        index++;  
        return tab[index - 1];  
    }  
}
```

Table des Matières

- **Interfaces**

- Déclaration, Implémentation
- Polymorphisme, héritage
- Comparaison d'objets
- Interfaces fonctionnelles, lambdas
- Gestion d'événements
- Itérateur

- **Classes abstraites**

- Application : timer

- **Classes internes**

- Classes internes et imbriquées
- Classes locales et anonymes

Classes Abstraites

- **Introduction**

- Une **classe concrète** implémente toutes les méthodes qu'elle déclare.
- Par opposition, une **classe abstraite** est une classe qui ne fournit pas l'implémentation d'au moins une de ses méthodes.
 - méthode définies par une interface partiellement implémentée
 - méthodes abstraites déclarées dans la classe ou un de ses parents avec le modificateur **abstract**
- Une classe abstraite doit être déclarée avec le modificateur **abstract**.
 - ne peut pas être instanciée;
 - est utilisée comme parent d'autres classes.

Classes Abstraites

- Exemple

```
public abstract class CompteBancaire {  
    public final String proprio;  
    private double solde;  
  
    public CompteBancaire(String proprio) {  
        this.proprio = proprio;  
    }  
  
    public double getSolde() {  
        return solde;  
    }  
  
    public void retirer(double montant) {  
        solde -= montant;  
    }  
  
    public void deposer(double montant) {  
        solde += montant;  
    }  
  
    public abstract void traitementMensuel();  
}
```

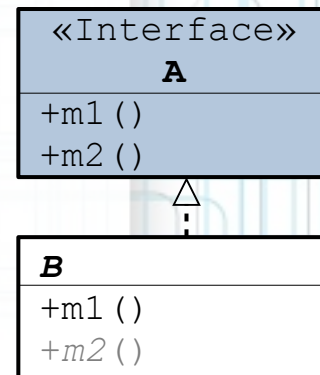
pas d'implémentation

Classes Abstraites

- Exemple

```
public interface I {  
    public void m1 ();  
    public void m2 ();  
}
```

```
public abstract class A implements I {  
    public void m1 () {  
        System.out.println("Youhou, je suis implémentée");  
    }  
}
```



L'interface `I` définit 2 méthodes.
La classe `A` n'en définit qu'une.

Classes Abstraites

- **Classes Abstraites vs Interfaces**

- Quelle sont les différences entre interfaces et classes abstraites ?
 - Il n'est pas possible d'hériter de plusieurs classes abstraites car l'héritage multiple de classes n'est pas permis en Java. C'est possible pour les interfaces.
 - Les classes abstraites peuvent définir des variables. Les interfaces ne peuvent définir que des constantes (les modificateurs **public static final** y sont implicites).
 - Les classes abstraites peuvent aussi définir des méthodes « concrètes ». Les interfaces ne définissent que des signatures de méthodes (*).
 - Les classes abstraites peuvent déclarer des méthodes abstraites non publiques. Les interfaces ne définissent que des signatures de méthodes publiques (**public** est implicite).

(*) depuis Java 8 et 9, des méthodes par défaut et privées sont possibles.

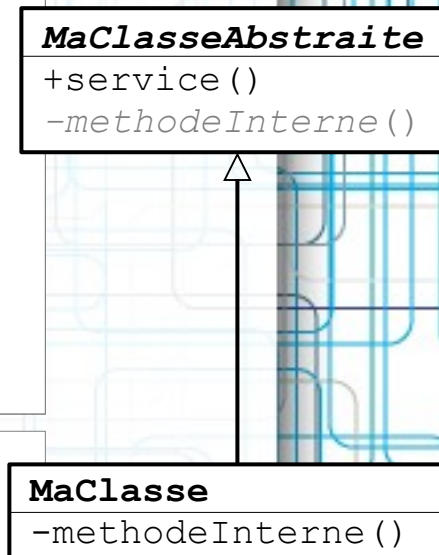
Classes Abstraites

- **Classes Abstraites vs Interfaces**

- Exemple : une interface ne permet pas de déclarer une méthode protégée alors qu'une classe abstraite le peut.

```
public abstract class MaClasseAbstraite {
    protected abstract void methodeInterne();
    public void service() {
        /* ... */
        methodeInterne();
        /* ... */
    }
}
```

```
public class MaClasse extends MaClasseAbstraite {
    @Override
    protected void methodeInterne() {
        ...
    }
}
```



Classes Abstraites

- **Classes Abstraites vs Interfaces**

- Les classes abstraites et les interfaces sont parfois utilisées en tandem, selon l'organisation suivante:
 - L'**interface** spécifie les méthodes à supporter
 - La classe abstraite implémente partiellement les comportements communs aux sous-classes
 - Les sous-classes implémentent complètement l'interface

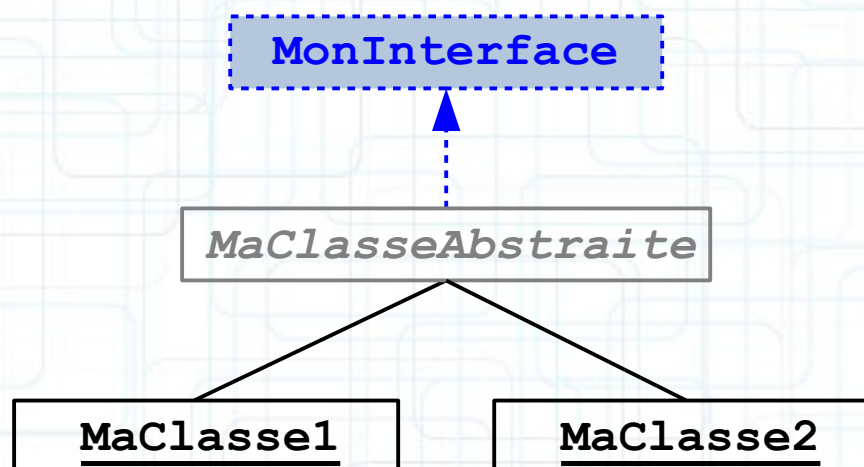


Table des Matières

- **Interfaces**

- Déclaration, Implémentation
- Polymorphisme, héritage
- Comparaison d'objets
- Interfaces fonctionnelles, lambdas
- Gestion d'événements
- Itérateur

- **Classes abstraites**

- Application : timer

- **Classes internes**

- Classes internes et imbriquées
- Classes locales et anonymes

Timer

- **Introduction**

- Un **timer** permet de déclencher l'exécution d'une action après l'écoulement d'un certain temps ou à intervalle régulier.
- Il existe plusieurs types de *timers* dans la bibliothèque Java. Un exemple est la classe `java.util.Timer` qui repose sur l'utilisation de classes abstraites.
- **Note** : La bibliothèque Java fournit également une gestion de timer dans la classe `javax.swing.Timer`. Cet autre timer repose sur l'utilisation d'interfaces et est plus approprié dans un contexte d'interface graphique.

Timer

- **Classe Timer**

- La classe `Timer` (`java.util`) fournit notamment les deux méthodes montrées ci-dessous.

```
public class Timer {  
    public void schedule(TimerTask t, long delay);  
    public void schedule(TimerTask t, long delay, long period);  
    /* ... */  
}
```

- La méthode `schedule()`
 - permet d'exécuter une action après un temps donné.
 - éventuellement, répéter l'action à intervalle régulier.
- Les tâches à exécuter sont définies sous la forme d'instances d'une sous-classe de la classe abstraite `TimerTask`.

Timer

- **Classe TimerTask**

TimerTask

```
+run()  
+cancel()  
+scheduledExecutionTime()
```

- La classe abstraite TimerTask définit plusieurs méthodes.

- action programmée, définie sous forme d'une méthode abstraite qu'il faut surcharger

```
public abstract void run();
```

- annuler les exécutions suivantes de la tâche

```
public void cancel();
```

- récupérer le temps de la prochaine exécution. Ce temps est mesuré en nombre de millisecondes écoulées depuis le 1er janvier 1970 minuit UTC.

```
public long scheduledExecutionTime();
```

Timer

- Exemple

```
public class MyTimerTask extends TimerTask {
    public void run() {
        System.out.println("action exécutée " +
            System.currentTimeMillis());
    }
}
```

```
Timer t = new Timer();
TimerTask task = new MyTimerTask();
System.out.println(System.currentTimeMillis());
t.schedule(task, 1000);
System.out.println(task.scheduledExecutionTime());
```

$t = 0$

```
1268056528480
1268056529480
```

$t = 1000\text{ms}$

```
action exécutée 1268056529480
```

```
t.schedule(new MyTimerTask(), 0, 5000);
```

```
action exécutée 1268058555198
action exécutée 1268058560198
action exécutée 1268058565198
action exécutée 1268058570198
action exécutée 1268058575198
...
```

```
TimerTask
+run()
+cancel()
(...)
```

```
MyTimerTask
+run()
```

Récupère le temps actuel
(ms à partir du 1/1/1970
minuit UTC)

Programme la tâche
dans 1000 ms

Programme la tâche
immédiatement pour
une exécution toutes
les 5000 ms

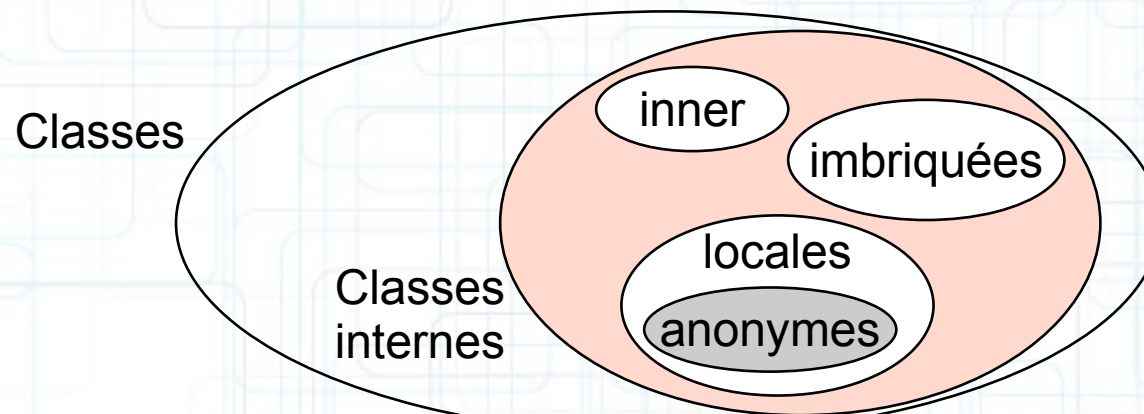
Table des Matières

- **Interfaces**
 - Déclaration, Implémentation
 - Polymorphisme, héritage
 - Comparaison d'objets
 - Interfaces fonctionnelles, lambdas
 - Gestion d'événements
 - Itérateur
- **Classes abstraites**
 - Application : timer
- **Classes internes**
 - Classes internes et imbriquées
 - Classes locales et anonymes

Classes internes

- **Introduction**

- Une **classe interne** est une classe définie à l'intérieur d'une autre classe.
- On distingue
 - **classe interne** (*inner class*): définie dans le corps d'une autre classe
 - **classe imbriquée** (*nested class*) : définie avec **static** dans le corps d'une autre classe
 - **classe locale** : définie dans une méthode
 - **classe anonyme** : classe locale sans nom



Classes internes

- **Introduction**

- Pourquoi s'intéresser aux classes internes ?
 - Syntaxe de déclaration et propriétés légèrement différentes de celles des classes que nous avons définies jusqu'ici.
- Les classes internes offrent les **avantages suivants**
 - L'utilisation de classes internes permet de renforcer l'encapsulation en « cachant » l'implémentation d'une classe de l'extérieur (notamment aux classes du même package).
 - Elles peuvent être définies de manière concise (voir classes anonymes)
 - Elles peuvent avoir accès à certaines données (arguments de méthodes et données privées).

Table des Matières

- **Interfaces**
 - Déclaration, Implémentation
 - Polymorphisme, héritage
 - Comparaison d'objets
 - Interfaces fonctionnelles, lambdas
 - Gestion d'événements
 - Itérateur
- **Classes abstraites**
 - Application : timer
- **Classes internes**
 - Classes internes et imbriquées
 - Classes locales et anonymes

Classes internes

- **Classe imbriquée (*nested*)**

- Une classe imbriquée est une classe interne définie avec le modificateur **static**. Il s'agit donc d'une **classe interne de classe**.
- Elle a accès aux membres de classe de sa classe englobante.

```
public class OuterClass {  
  
    private static int x = 3;  
    private static int y = 2;  
  
    public static class InnerClass {  
        private static int x = 5;  
        public void sayHello() {  
            System.out.println("Hello");  
            System.out.println(x + "," + OuterClass.x + "," + y);  
        }  
    }  
}
```

l'attribut `x` de la classe englobante est à portée, mais masqué

```
var i = new OuterClass.InnerClass();  
i.sayHello(); /* 5,3,2 */
```

Pour référer au nom de la classe imbriquée à l'extérieur de sa classe englobante, il faut fournir son nom complet.

Classes internes

- **Classe interne (*inner*)**

- Une classe interne « *inner* » est une classe interne définie sans le modificateur **static**. C'est une **classe interne d'instance**.
- Une instance d'une telle classe ne peut exister sans une instance de sa classe englobante. Elle a accès aux membres de la classe englobante, y compris les membres d'instance.

```
public class OuterClass {  
    protected int x = 3;  
    public class InnerClass {  
        public void sayHello() {  
            int x = 4;  
            System.out.println("Hello");  
            System.out.println(x + "," + Outerclass.this.x);  
        }  
    }  
}
```

```
var o = new OuterClass();  
var i = o.new InnerClass();  
i.sayHello(); /* 4,3 */
```

Classes internes

- **Poupées russes**

- Il est possible de définir une classe interne ou imbriquée à l'intérieur d'une autre classe interne ou imbriquée.

```
public class OuterClass { ..... ➡ Classe la plus externe  
    (top-level class)  
    public class InnerClass {  
        public class InnerInnerClass {  
            public class InnerInnerInnerClass {  
            }  
        }  
    }  
}
```



Classes internes

- **Exemple d'utilisation**

- Variante sauce classe interne du Timer...

```
import java.util.*;

public class OuterClass {

    private static class MyTimerTask extends TimerTask {
        public void run() {
            System.out.println("action exécutée " +
                               System.currentTimeMillis());
        }
    }

    public static void main(String[] args) {
        Timer t= new Timer();
        TimerTask task= new MyTimerTask();
        System.out.println(System.currentTimeMillis());
        t.schedule(task, 1000);
        System.out.println(task.scheduledExecutionTime());
    }
}
```

Table des Matières

- **Interfaces**
 - Déclaration, Implémentation
 - Polymorphisme, héritage
 - Comparaison d'objets
 - Interfaces fonctionnelles, lambdas
 - Gestion d'événements
 - Itérateur
- **Classes abstraites**
 - Application : timer
- **Classes internes**
 - Classes internes et imbriquées
 - Classes locales et anonymes

Classes internes

- **Classe locale**

- Une classe déclarée dans une méthode est une **classe locale**.
 - Pas de spécificateur d'accès
 - Portée limitée à la méthode; le nouveau type ainsi défini n'est pas visible en dehors de la méthode
 - Accès aux variables locales (si « *effectivement* » **finales**)
- Illustration à l'aide d'un... Timer (quelle imagination !)

```
public void timerExample() {  
    class MyTimerTask extends TimerTask {  
        @Override  
        public void run() {  
            System.out.println("action exécutée " +  
                System.currentTimeMillis());  
        }  
    }  
    (new Timer()).schedule(new MyTimerTask(), 1000, 10000);  
}
```

Classes internes

- **Classe locale**

- Dans la variante qui suit, une instance de `MyTimerTask`, une classe locale, est retournée par la méthode.

```
public TimerTask createTimerTask() {  
    class MyTimerTask extends TimerTask {  
        @Override  
        public void run() {  
            System.out.println("action exécutée" +  
                System.currentTimeMillis());  
        }  
    }  
    return new MyTimerTask();  
}
```

```
(new Timer()).schedule(createTimerTask(), 1000, 10000);
```

- Le type `MyTimerTask` n'étant pas connu en dehors de la méthode, on utilise le polymorphisme : le type de retour est `TimerTask`.

Classes internes

- **Classe anonyme**

- Moyen d'exprimer brièvement une classe locale : déclaration de classe et instantiation combinées.
 - Se base sur une classe parent (explicitement) ou une interface
 - Pas de déclaration de constructeur
 - Accès aux variables locales (si « *effectivement* » **finales**)
- Utilisation du `Timer` revisitée (une fois de plus).

```
public static void main(String[] args) {  
    var task = new TimerTask() {  
        public void run() {  
            System.out.println("action exécutée");  
        }  
    };  
    (new Timer()).schedule(task, 1000);  
}
```

→ Classe parent

Classes internes

- **Classe anonyme**

- Fortement utilisé dans les interfaces graphiques pour la réaction à des événements

```
private JPanel createPanel() {  
    JPanel thePanel = new JPanel();  
    thePanel.setLayout(  
        new BoxLayout(thePanel, BoxLayout.PAGE_AXIS));  
    JButton theButton = new JButton("Click me!");  
    JLabel theLabel = new JLabel("Boring label");  
    thePanel.add(theButton);  
    thePanel.add(theLabel);  
    theButton.addActionListener(new ActionListener() {  
        public void actionPerformed(ActionEvent e) {  
            theLabel.setText("I've been pressed");  
        }  
    });  
    return thePanel;  
}
```

Interface

